GUÍA PRÁCTICA LAB 04 Y PROMPTS OPTIMIZADOS • EDICIÓN RETAIL OMNICANAL (RETAIL PULSE)

Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

TRYOLABS • GOOGLE ANTIGRAVITY WORKSHOP • LAB 04 (ES)

Lab 04: Capstone Multi-Agente — Google ADK, Protocolo A2A, Enrutamiento Omnicanal y Despliegue en Gemini Enterprise

Duración: 45 Minutos | **Perfil**: Track Técnico y Comercial | **Prerrequisitos**: Labs 01, 02 y 03 Completados

Idioma / Language: Español | English • **Guía de Prompts:** PROMPTS.es.md | PROMPTS.md • **PDF:** ES | EN

1. Escenario Retail y Contexto de Negocio

En los Labs 01–03, tu equipo estabilizó los pisos de margen promocional del 15%, reparó el cálculo de inventario True ATP, codificó los estándares de cumplimiento retail (**`GEMINI.md`**, **`.agents/rules/retail-standards.md`**, **`.agents/skills/retail-otif-margin-auditor/SKILL.md`**), y conectó Antigravity a **`retail_pulse.db`** mediante FastMCP y hooks deterministas **`PreToolUse`**.

Sin embargo, persisten dos desafíos operativos clave:

1. **35 órdenes VIP (niveles de lealtad `DIAMOND` y `GOLD`)** que representan **$749,300.00 en Customer Lifetime Value (LTV)** siguen bloqueadas en estado **`AT_RISK_STOCKOUT`** porque su tienda insignia original no tenía suficiente inventario True ATP.
2. Las operaciones retail empresariales requieren **agentes de IA especializados** — un **Agente Técnico WMS/POS de Inventario** y un **Agente Comercial de Merchandising, Margen y OTIF** — que se coordinen mediante **Google ADK (Agent Development Kit)**, expongan un endpoint abierto bajo el protocolo **A2A (Agent2Agent Protocol)** (**`/.well-known/agent.json`** y **`POST /api/v1/a2a/tasks/send`**), puedan probarse localmente en la interfaz **ADK Web UI (`adk web`)**, y se desplieguen en **Google Cloud Run + Gemini Enterprise** usando **Terraform**.

2. Objetivos de Aprendizaje

En este laboratorio capstone:

1. Usarás **`/plan`**, **`/boost`** y **`/teamwork`** para implementar **`optimize_at_risk_order_routing`** y **`claim_vendor_otif_chargebacks`** en **`src/services/fulfillment_router.py`**:

- **`optimize_at_risk_order_routing`**: Re-enruta las 35 órdenes VIP **`AT_RISK_STOCKOUT`** al nodo de cumplimiento con mayor margen neto post-envío y **`true_atp >= qty`** (priorizando Centros de Distribución **`DC`** sobre tiendas **`STORE`** en caso de empate), cambiando su estado a **`REROUTED_SAVED`** y protegiendo **`$749,300.00`** en LTV VIP.

- **`claim_vendor_otif_chargebacks`**: Audita los contratos de proveedores en incumplimiento y actualiza los 6 contratos (**`actual_otif_pct < contracted_otif_pct`**) a **`CHARGEBACK_CLAIMED`**, recuperando **`$98,250.00`** en notas de débito contractuales.

2. Diseñarás e implementarás un **Sistema Multi-Agente con Google ADK y el Protocolo A2A** en **`src/agents/adk_a2a_orchestrator.py`**:

- **`WMSInventoryATPAgent`** (**`wms_inventory_atp_agent`**): Agente técnico WMS/POS que audita el True ATP y ejecuta el re-enrutamiento óptimo omnicanal.
- **`MerchandisingMarginOTIFAgent`** (**`merchandising_margin_otif_agent`**): Agente comercial de Merchandising y Cumplimiento de Proveedores que aplica el piso de margen bruto del 15% y reclama **`$98,250.00`** en penalidades OTIF.
- **`RetailPulseOrchestratorAgent`**: Orquestador raíz (**`run_multi_agent_workflow`**) que coordina los handoffs A2A entre el agente técnico WMS y el agente comercial de Merchandising.
- **A2A Discovery Card** (**`get_a2a_agent_card`** para **`GET /.well-known/agent.json`** y **`GET /api/v1/a2a/agent-card`**) y **Endpoint A2A JSON-RPC 2.0** (**`handle_a2a_jsonrpc`** para **`POST /api/v1/a2a`** y **`POST /api/v1/a2a/tasks/send`** soportando **`tasks/send`**, **`message/send`** y **`tasks/get`**).

3. Probarás la jerarquía multi-agente interactivamente en tu máquina local usando **ADK Development UI (`adk web`)** mediante **`./scripts/run_adk_web.sh`** (**`adk_agents/retail_pulse/agent.py`**).
4. Validarás la infraestructura de **Google Cloud Run + Vertex AI + Gemini Enterprise** con **Terraform** (**`infra/terraform/`**) y **`./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run`**.

3. Diagrama de Arquitectura: ADK + A2A + Cloud Run + Gemini Enterprise

4. Instrucciones Paso a Paso

Paso 1: Inicializar el Estado Inicial del Lab 04

Ejecuta el instalador del Lab 04 para aplicar automáticamente las soluciones de los Labs 01, 02 y 03 y cargar las plantillas iniciales del Lab 04 (**`src/services/fulfillment_router.py`** y **`src/agents/adk_a2a_orchestrator.py`**):

```
./labs/lab-04-multi-agent-capstone/install.sh
```

Ejecuta el script de verificación para observar las pruebas pendientes del Lab 04 (**`NotImplementedError`** en **`fulfillment_router.py`** y **`adk_a2a_orchestrator.py`**):

```
./labs/lab-04-multi-agent-capstone/verify.sh
```

Paso 2: Implementar el Motor de Enrutamiento Omnicanal y Penalidades OTIF (`src/services/fulfillment_router.py`)

Abre **`src/services/fulfillment_router.py`** e implementa:

1. **`optimize_at_risk_order_routing(loyalty_tiers=("DIAMOND", "GOLD"), persist_changes=True, db_path=None) -> dict[str, Any]`**:

- Consulta **`omnichannel_orders`** unido con **`sku_catalog`** para órdenes en **`loyalty_tiers`** ordenadas por **`customer_ltv_usd DESC, order_id ASC`**.

- Identifica las órdenes donde **`status == "AT_RISK_STOCKOUT"`** o donde **`evaluate_location_atp(..., required_qty=qty)["oversell_risk"]`** en el nodo actual sea **`True`**.
- Busca ubicaciones candidatas en **`wms_inventory`** para el SKU donde **`evaluate_location_atp(..., required_qty=qty)["can_fulfill"]`** sea **`True`**.
- Calcula **`shipping_cost = float(loc["shipping_zone_cost_usd"]) * qty`**, **`post_shipping_margin_usd = round((unit_price - unit_cost) * qty - shipping_cost, 2)`** y **`post_shipping_margin_pct`**, ordenando los candidatos por **`(-c["post_shipping_margin_usd"], 0 if c["location_type"] == "DC" else 1, -c["true_atp"])`**.
- Cuando **`persist_changes=True`**, actualiza **`omnichannel_orders`** (**`assigned_location_id = best["location_id"]`**, **`shipping_cost_usd = best["shipping_cost_usd"]`**, **`status = "REROUTED_SAVED"`**).
- Devuelve **`at_risk_orders_evaluated`**, **`rerouted_orders_count`** (**`35`**), **`vip_ltv_protected_usd`** (**`749300.0`**), **`post_shipping_margin_saved_usd`** y **`rerouted_orders`**.

2. **`claim_vendor_otif_chargebacks(min_shortfall_pct=0.01, db_path=None) -> dict[str, Any]`**:

- Llama a **`audit_all_vendor_otif_contracts(only_breaching=True, db_path=db_path)`**, filtra contratos con **`shortfall_pct >= min_shortfall_pct`**, actualiza **`vendor_otif_contracts`** (**`status = 'CHARGEBACK_CLAIMED'`**) y devuelve **`contracts_claimed_count`** (**`6`**), **`total_chargeback_claimed_usd`** (**`98250.0`**) y **`claimed_contracts`**.

Paso 3: Implementar el Orquestador Multi-Agente Google ADK y A2A (`src/agents/adk_a2a_orchestrator.py`)

Abre **`src/agents/adk_a2a_orchestrator.py`** y revisa cómo el sistema multi-agente conecta los perfiles **Técnico** y **Comercial**:

1. **`WMSInventoryATPAgent.execute(task_payload)`**:

- Ejecuta **`optimize_at_risk_order_routing(...)`** y **`get_inventory_atp_summary(...)`**, devolviendo **`agent_id`**, **`role`**, **`atp_network_summary`**, **`routing_optimization`** y **`handoff_recommendation`**.

2. **`MerchandisingMarginOTIFAgent.execute(task_payload, wms_routing_output=None)`**:

- Consume el resultado del enrutamiento WMS, ejecuta **`remediate_margin_floor_violations(...)`** (aplicando el piso de margen bruto del 15% en las 28 órdenes **`MARGIN_HOLD`**) y ejecuta **`claim_vendor_otif_chargebacks(...)`** (recuperando **`$98,250.00`** en 6 proveedores en incumplimiento).

3. **`RetailPulseOrchestratorAgent.run_multi_agent_workflow(prompt, skill_id=None, metadata=None)`**:

- Coordina **`WMSInventoryATPAgent`** -> **`MerchandisingMarginOTIFAgent`** mediante **`a2a_handoffs`**, construye los metadatos de Google ADK (**`adk_runtime`**) y devuelve el impacto financiero combinado (**`vip_ltv_protected_usd = 749300.0`**, **`otif_chargebacks_claimed_usd = 98250.0`**, **`margin_recovered_usd = 18420.5`**).

4. **Funciones del Protocolo A2A (`get_a2a_agent_card` y `handle_a2a_jsonrpc`)**:

- **`GET /.well-known/agent.json`** y **`GET /api/v1/a2a/agent-card`**: Devuelve la Agent Card A2A (**`protocolVersion: "0.2.1"`**) declarando las habilidades **`wms_atp_rerouting`**, **`promo_margin_guard_audit`** y **`vendor_otif_chargeback_recovery`**.
- **`POST /api/v1/a2a`** y **`POST /api/v1/a2a/tasks/send`**: Procesa peticiones JSON-RPC 2.0 **`tasks/send`**, **`message/send`** y **`tasks/get`** y retorna objetos A2A Task completados con **`artifacts`**.

Paso 4: Probar Localmente con ADK Development UI (adk web) y el Command Center FastAPI

Google ADK incluye una interfaz web interactiva (**`adk web`**) para inspeccionar llamadas a herramientas, transferencias entre sub-agentes y el estado de sesión localmente antes de desplegar en Cloud Run.

1. Verifica la configuración de ADK Web en modo check/dry-run:

   ```
   ./scripts/run_adk_web.sh --check
   ```

2. Inicia la interfaz interactiva ADK Development UI en el puerto **`8080`**:

   ```
   ./scripts/run_adk_web.sh --port 8080
   ```

3. Abre **`http://127.0.0.1:8080`** en tu navegador (o mediante **`/browser`**), selecciona el agente **`retail_pulse`** en el menú superior izquierdo y prueba prompts como:

- *"Ejecuta un triaje omnicanal completo abarcando el enrutamiento True ATP en WMS, los pisos de margen promocional del 15% y los chargebacks OTIF de proveedores."*
- *"Consulta el True ATP en todos los nodos para SKU-OLED-65 y explica el riesgo de sobreventa VIP."*

4. Inicia el **Retail Pulse FastAPI Command Center** en el puerto **`8002`**:

   ```
   ./.venv/bin/uvicorn src.main:app --host 127.0.0.1 --port 8002
   ```

Abre **`http://localhost:8002`** y **`http://localhost:8002/.well-known/agent.json`** para inspeccionar el dashboard en vivo y la Agent Card A2A.

Paso 5: Validar el Pipeline de Despliegue en Google Cloud Run + Terraform + Gemini Enterprise

El repositorio incluye **Infraestructura como Código con Terraform** (**`infra/terraform/`**), un **`Dockerfile`** multi-etapa para Cloud Run y scripts de despliegue (**`deploy/deploy_gcp.sh`** y **`deploy/register_gemini_enterprise_agent.sh`**).

1. **Prueba el pipeline de Cloud Run + Terraform + Gemini Enterprise en modo `--dry-run`** (no requiere credenciales de GCP):

   ```
   ./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run --project-id mi-proyecto-r \
   etail-gcp --region us-central1 --gemini-app-id retail-pulse-enterprise-app
   ```

2. **Opcional — Despliega en vivo en Google Cloud Platform** (autenticado con **`gcloud auth login`**):

   ```
   ./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --project-id TU_GCP_PROJECT_ID --reg \
   ion us-central1 --gemini-app-id TU_GEMINI_ENTERPRISE_APP_ID
   ```

5. Lista de Verificación

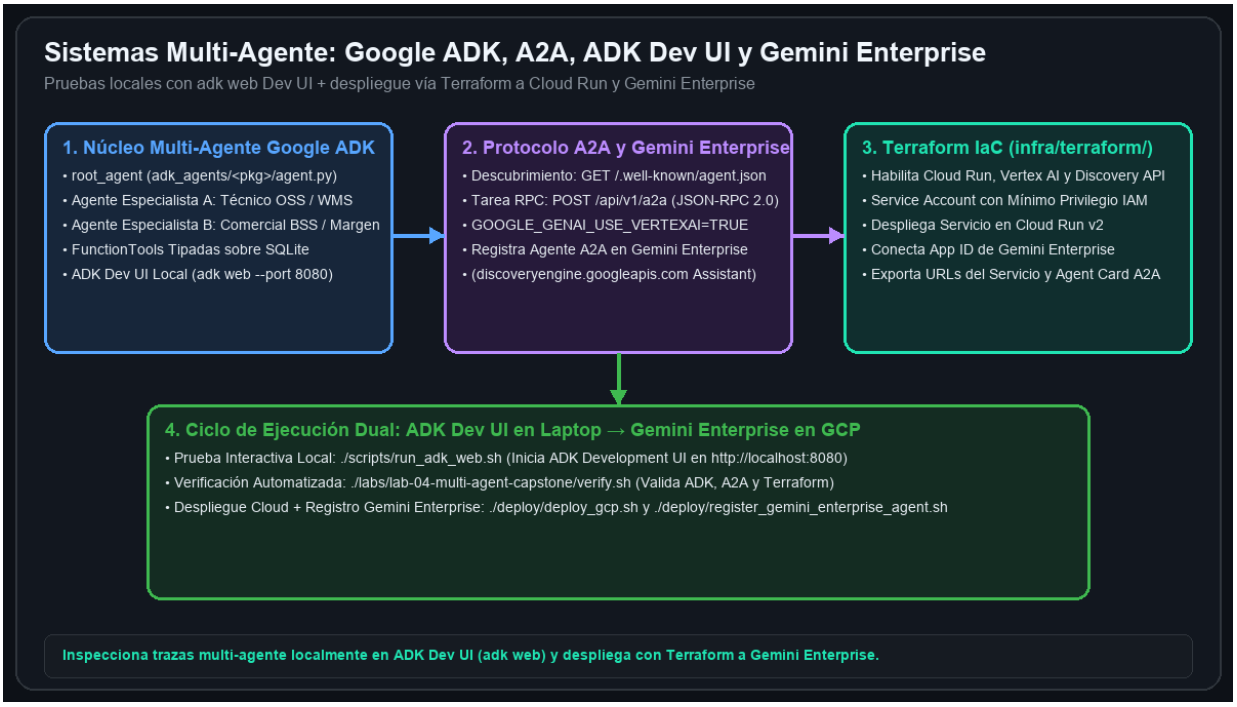
Ejecuta el script de verificación completo del capstone (que corre las **20 pruebas automatizadas** de los Labs 01–04):

```
./labs/lab-04-multi-agent-capstone/verify.sh
```

- [ ] **`./labs/lab-04-multi-agent-capstone/verify.sh`** aprueba las **20/20 pruebas automatizadas** en **`test_lab01_triage.py`** (6), **`test_lab02_rules_skills.py`** (4), **`test_lab03_mcp_hooks.py`** (3) y **`test_lab04_capstone.py`** (7).
- [ ] Las 35 órdenes VIP en **`AT_RISK_STOCKOUT`** pasan a **`REROUTED_SAVED`** protegiendo **`$749,300.00`** en LTV VIP, y los 6 contratos de proveedores en incumplimiento pasan a **`CHARGEBACK_CLAIMED`** recuperando **`$98,250.00`**.

- [ ] **`GET /.well-known/agent.json`** devuelve una Agent Card A2A válida y **`POST /api/v1/a2a/tasks/send`** ejecuta el handoff multi-agente ADK (**`wms_inventory_atp_agent -> merchandising_margin_otif_agent`**).
- [ ] **`./scripts/run_adk_web.sh --check`** y **`./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run`** finalizan exitosamente.

¿Necesitas aplicar directamente la solución verificada del capstone? Ejecuta **`./labs/lab-04-multi-agent-capstone/apply_solution.sh`**.



LAB 04 PROMPTS OPTIMIZADOS PARA COPIAR Y PEGAR (ES)
Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

Playbook de Prompts del Lab 04: Google ADK, Protocolo A2A, Enrutamiento Omnicanal y Despliegue en Gemini Enterprise

Idioma / Language: Español | English • **Guía del Lab:** README.es.md | README.md • **PDF:** ES | EN

Copia y pega estos prompts en **Google Antigravity** mientras completas el Capstone del Lab 04. Cada fase incluye prompts tanto para el **Track Técnico** como para el **Track Comercial / Ejecutivo**.

Fase 1: Descubrimiento y Planificación Multi-Agente (`/plan` y `/boost`)

Prompt 1A — Track Técnico: Plan Arquitectónico para Enrutamiento + ADK + A2A (`/plan`)

```
/plan Lee @src/services/fulfillment_router.py, @src/agents/adk_a2a_orchestrator.py, @adk \
  _agents/retail_pulse/agent.py, @src/main.py y @tests/test_lab04_capstone.py.
Crea un plan de implementación paso a paso para completar nuestro Capstone del Lab 04:
1. En `src/services/fulfillment_router.py`, implementar `optimize_at_risk_order_routing` \
   y `claim_vendor_otif_chargebacks` para clasificar los nodos elegibles (`true_atp >= q \
  ty`) por `(-post_shipping_margin_usd, 0 if location_type == 'DC' else 1, -true_atp)`,  \
  re-enrutar las 35 órdenes VIP `AT_RISK_STOCKOUT` a `REROUTED_SAVED` (protegiendo $749, \
  300.00 en LTV VIP), y reclamar $98,250.00 en los 6 contratos OTIF en incumplimiento (` \
  CHARGEBACK_CLAIMED`).
2. En `src/agents/adk_a2a_orchestrator.py`, implementar:
   - `WMSInventoryATPAgent.execute(task_payload)` (resumen de red True ATP + re-enrutami \
  ento óptimo omnicanal + recomendación de handoff)
   - `MerchandisingMarginOTIFAgent.execute(task_payload, wms_routing_output)` (remediaci \
  ón del piso de margen bruto del 15% + recuperación de penalidades OTIF de proveedores)
   - `RetailPulseOrchestratorAgent.run_multi_agent_workflow(prompt, skill_id, metadata)` \
   coordinando `WMSInventoryATPAgent` -> `MerchandisingMarginOTIFAgent` con metadatos de \
   Google ADK
   - `get_a2a_agent_card(base_url)` y `handle_a2a_jsonrpc(payload, db_path)` para `GET / \
  .well-known/agent.json` y `POST /api/v1/a2a/tasks/send` (`tasks/send`, `message/send`  \
  y `tasks/get`).
```

Prompt 1B — Track Comercial / Operaciones: Análisis de Gobernanza Multi-Agente (`/boost`)

```
/boost Lee @src/services/fulfillment_router.py, @src/agents/adk_a2a_orchestrator.py y @t \
  ests/test_lab04_capstone.py.
Desde una perspectiva de rentabilidad omnicanal retail, analiza por qué separar responsa \
  bilidades entre un agente técnico `WMSInventoryATPAgent` (True ATP y enrutamiento por  \
  margen neto post-envío priorizando Centros de Distribución `DC`) y un agente comercial \
   `MerchandisingMarginOTIFAgent` (piso de margen bruto del 15% y notas de débito OTIF)  \
  conectados mediante el Protocolo A2A previene tanto la fuga de margen negativo como la \
```

```
s cancelaciones de clientes VIP. Luego implementa ambos módulos para que pasen las 20  \
pruebas del taller.
```

Fase 2: Ejecución Multi-Agente (`/teamwork` y Subagentes)

Prompt 2A — Orquestación Paralela de Subagentes (`/teamwork`)

```
/teamwork Coordina tres flujos de trabajo especializados para completar el Capstone del  \
  Lab 04:
1. Ingeniero de Enrutamiento Omnicanal: Implementa `optimize_at_risk_order_routing` y `c \
  laim_vendor_otif_chargebacks` en @src/services/fulfillment_router.py para que las 35 ó \
  rdenes VIP `AT_RISK_STOCKOUT` se re-enruten a `REROUTED_SAVED` ($749,300.00 LTV VIP pr \
  otegido) y los 6 contratos de proveedores en incumplimiento pasen a `CHARGEBACK_CLAIME \
  D` ($98,250.00 recuperados).
2. Arquitecto de Sistemas ADK y A2A: Implementa `WMSInventoryATPAgent`, `MerchandisingMa \
  rginOTIFAgent`, `RetailPulseOrchestratorAgent`, `get_a2a_agent_card` y `handle_a2a_jso \
  nrpc` en @src/agents/adk_a2a_orchestrator.py.
3. Auditor de QA, ADK Web y Cloud Run: Ejecuta `./labs/lab-04-multi-agent-capstone/verif \
  y.sh` para validar las 20 pruebas de los Labs 01-04, verifica `./scripts/run_adk_web.s \
  h --check` y `./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run`, y genera un \
  a tabla ejecutiva con la recuperación financiera total.
```

Fase 3: Edición en Línea (`Cmd+I` / `Ctrl+I`)

Abre **`src/services/fulfillment_router.py`**, selecciona **`optimize_at_risk_order_routing`** y **`claim_vendor_otif_chargebacks`**, presiona **`Cmd+I`** (macOS) o **`Ctrl+I`** (Windows/Linux) e ingresa:

```
Implementa `optimize_at_risk_order_routing` usando `evaluate_location_atp` para filtrar  \
  nodos candidatos con `can_fulfill == True`, calcula `post_shipping_margin_usd = round( \
  (unit_price - unit_cost) * qty - shipping_cost, 2)`, ordena por `(-post_shipping_margi \
  n_usd, 0 if location_type == 'DC' else 1, -true_atp)` y actualiza `omnichannel_orders` \
   (`status='REROUTED_SAVED'`) cuando `persist_changes=True`. Luego implementa `claim_ve \
  ndor_otif_chargebacks` usando `audit_all_vendor_otif_contracts(only_breaching=True)` p \
  ara cambiar el estado de los contratos en incumplimiento a `status='CHARGEBACK_CLAIMED \
  '`.
```

Fase 4: Pruebas Interactivas Locales con `adk web` y `/browser`

Prompt 4A — Probar la Jerarquía Multi-Agente Localmente en ADK Web UI (`adk web`)

```
Valida nuestra configuración local de Google ADK Development UI ejecutando `./scripts/ru \
  n_adk_web.sh --check`, e inspecciona @adk_agents/retail_pulse/agent.py para confirmar  \
  cómo `root_agent` (`retail_pulse_orchestrator`) delega entre `wms_inventory_atp_agent` \
   y `merchandising_margin_otif_agent`.
```

Prompt 4B — Verificación Visual de Punta a Punta en el Navegador (`/browser`)

```
/browser Inicia el servidor FastAPI (`./.venv/bin/uvicorn src.main:app --host 127.0.0.1  \
  --port 8002`) si aún no está corriendo, abre `http://localhost:8002` y verifica:
1. El encabezado muestra el logo de Tryolabs ("Delivered by Tryolabs") y los KPIs del Re \
  tail Pulse Command Center.
2. Ejecuta las acciones de remediación (`POST /api/v1/remediate/margins`, `POST /api/v1/ \
```

```
  remediate/routing` o `POST /api/v1/a2a/tasks/send`).
3. Confirma que Margin Hold Orders = 0, VIP Stockout Risk = 0 ($749,300.00 LTV VIP prote \
  gido), y que los 6 contratos de proveedores en incumplimiento muestran `CHARGEBACK_CLA \
  IMED` sumando $98,250.00.
4. Consulta también `http://localhost:8002/.well-known/agent.json` para inspeccionar la  \
  Agent Card A2A en vivo.
```

Fase 5: Despliegue en Cloud Run + Terraform y Registro en Gemini Enterprise

Prompt 5 — Validar Infraestructura Terraform y Registro A2A en Gemini Enterprise

```
Lee @infra/terraform/main.tf, @deploy/deploy_gcp.sh y @deploy/register_gemini_enterprise \
  _agent.sh.
Ejecuta `./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run --project-id retail- \
  pulse-prod --region us-central1 --gemini-app-id retail-pulse-ge-app` y explica cada pa \
  so del pipeline de despliegue:
1. Construcción de la imagen de contenedor con Cloud Build y Artifact Registry
2. Aprovisionamiento de Cloud Run v2, cuenta de servicio IAM de mínimo privilegio y perm \
  isos de Vertex AI con Terraform
3. Registro de la Agent Card A2A de Cloud Run (`/.well-known/agent.json` y `/api/v1/a2a/ \
  tasks/send`) en Gemini Enterprise (Discovery Engine).
```